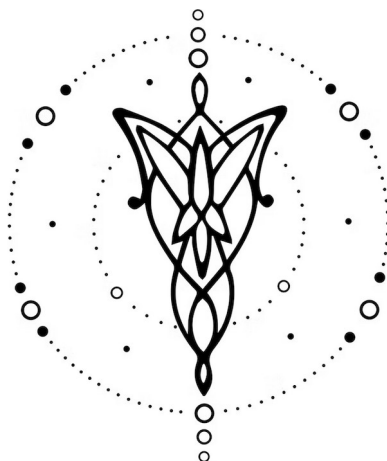




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de marzo de 2026

1. Equipo.....	1
2. Repositorio.....	1
3. Dominio.....	1
4. Construcciones.....	2
5. Casos de prueba.....	3
6. Ejemplos.....	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Alvaro	Capridi	64.206	acapridi@itba.edu.ar
Tiziano	Ametller	65.004	tametller@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/TizianoAmetller/Flex-Bison-Compiler>.

3. Dominio

Desarrollar un lenguaje que permita declarar *unidades*, agruparlas en *equipos* y definir *habilidades* para simular combates por turnos estilo RPG (*Role-Playing Game*). El lenguaje debe permitir enfrentar dos equipos en una batalla, resolver el orden de turnos, aplicar daño, curación y efectos de estado sobre las unidades, y registrar el desarrollo completo del combate.

Para reducir la complejidad del proyecto, el factor aleatorio se modela mediante una notación de dados nativa del lenguaje (*e.g.*, 2d6+3), y las habilidades operan exclusivamente sobre las unidades que participan de la batalla, sin necesidad de un motor gráfico o físico subyacente.

Toda la simulación ocurre dentro de la memoria del mismo programa; las unidades, equipos y batallas son completamente virtuales y no existe ningún tipo de renderizado ni interacción en tiempo real. El resultado de una batalla se expresa como un registro (*log*) textual de su desarrollo, turno a turno, hasta que uno de los dos equipos quede sin unidades con vida.

La implementación satisfactoria de este lenguaje permitiría a un equipo de diseño de videojuegos balancear rápidamente las estadísticas y habilidades de unidades y enemigos, iterando sobre numerosos combates simulados antes de integrar dicho contenido a un motor gráfico real, minimizando el tiempo de iteración y los costos de testeo manual.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear una o varias *unidades* (*unit*), y cada una tendrá nombre y atributos base: *hp* (vida), *atk* (ataque), *def* (defensa) y *spd* (velocidad).
- (II). Se podrán agrupar una o varias unidades dentro de un equipo, ya sea el bando aliado (*party*) o el bando rival (*encounter*).
- (III). Se podrán definir *habilidades* (*ability*) que se apliquen sobre una unidad objetivo (*target*), indicando el daño, curación o efecto que produce.
- (IV). El daño y la curación de una habilidad podrán expresarse combinando expresiones aritméticas con notación de dados (*e.g.*, 2d6+3, es decir, dos dados de 6 caras más 3).
- (V). Las variables podrán ser de tipo *unit*, *dice*, *integer*, *boolean* o *string*.
- (VI). Las variables podrán ser vectores de alguno de los tipos anteriores (*e.g.*, la lista de unidades de un equipo).
- (VII). Se proveerán operadores relacionales como *<*, *>*, *=*, *≠*, *≤* y *≥*.
- (VIII). Se proveerán operaciones aritméticas básicas como *+*, *-*, *** y */*.
- (IX). Se proveerán operaciones lógicas básicas como AND, OR y NOT.
- (X). Se proveerán estructuras de control básicas de tipo IF-THEN-ELSE, FOR y WHILE.
- (XI). Se podrán aplicar efectos de estado (*e.g.*, *poison*, *stun*) sobre una unidad, indicando su duración en turnos.
- (XII). Se podrá iniciar una batalla (*battle*) entre dos equipos, la cual resuelve automáticamente el orden de turnos según el atributo *spd* de cada unidad.
- (XIII). Se podrán emitir mensajes de registro (*log*) durante la resolución de una batalla, describiendo cada acción ejecutada.

Además, se prevé validar las siguientes **restricciones semánticas** durante el análisis:

- (XIV). Toda unidad referenciada en una habilidad o equipo debe haber sido declarada previamente.
- (XV). No puede existir más de una unidad con el mismo nombre dentro de la misma batalla.
- (XVI). Una habilidad no puede operar entre tipos incompatibles (*e.g.*, sumar un `dice` con un `string`).
- (XVII). Un `target` debe resolver siempre a una única unidad válida al momento de aplicarse la habilidad.
- (XVIII). Una batalla no puede iniciarse si alguno de los dos equipos está vacío.
- (XIX). Un efecto de estado no puede aplicarse dos veces en simultáneo sobre la misma unidad con la misma etiqueta.

El **modelo de resolución** de una batalla sigue el siguiente esquema, turno a turno:

- (XX). Se calcula el orden de turno de las unidades vivas según su atributo `spd` (de mayor a menor).
- (XXI). Se resuelve la habilidad de la unidad en turno sobre su `target`.
- (XXII). Se aplican y decrementan los efectos de estado activos sobre las unidades afectadas.
- (XXIII). Se verifica la condición de victoria: si alguno de los dos equipos quedó sin unidades con `hp` > 0 , la batalla finaliza.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que declare una unidad con sus cuatro atributos base.
- (II). Un programa que arme dos equipos, uno con una unidad y otro con dos unidades.
- (III). Un programa que defina una habilidad de daño fijo (sin dados) sobre un objetivo.
- (IV). Un programa que defina una habilidad de daño con notación de dados (*e.g.*, `1d6+2`).
- (V). Un programa que defina una habilidad de curación condicionada al `hp` actual del objetivo mediante IF-THEN-ELSE.
- (VI). Un programa que resuelva una batalla completa entre dos equipos de una unidad cada uno.
- (VII). Un programa que resuelva el orden de turnos entre unidades con distinta velocidad (`spd`).
- (VIII). Un programa que aplique un efecto de estado (*e.g.*, `poison`) con duración de varios turnos.
- (IX). Un programa que emita mensajes de *log* durante la resolución de una batalla.
- (X). Un programa que declare un equipo con más de dos unidades y las recorra con un bucle FOR.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que asigne la misma unidad a dos equipos distintos en simultáneo.
- (III). Un programa que referencie una unidad inexistente como objetivo de una habilidad.
- (IV). Un programa que intente iniciar una batalla con un equipo vacío.
- (V). Un programa que intente operar entre tipos de datos incompatibles (*e.g.*, sumar un `dice` con un `string`).

6. Ejemplos

En el **Cód. (6.1)**, se puede ver la declaración de dos unidades, su agrupación en equipos, y una habilidad de daño (*Slash*) aplicada en una batalla entre ambos.

```
// Declarar las unidades participantes:
unit Hero { hp: 30, atk: 8, def: 4, spd: 6 };
unit Goblin { hp: 12, atk: 5, def: 1, spd: 4 };

// Agrupar las unidades en sus respectivos equipos:
party Heroes = [Hero];
encounter Goblins = [Goblin];

// Definir una habilidad de daño con notación de dados:
ability Slash on target {
    deal 1d6 + atk to target;
}

// Iniciar la batalla entre ambos equipos:
battle Heroes vs Goblins;
```

Código 6.1: Combate entre un héroe y un goblin usando la habilidad *Slash*.

En el **Cód. (6.2)**, se define una habilidad de curación (*Heal*) que evalúa el hp actual del objetivo antes de decidir si aplica el efecto.

```
// Definir una habilidad de curación condicionada al HP del objetivo:
ability Heal on target {
    if (target.hp < 10) {
        heal 2d4 + 2 to target;
        log "{1} fue curado", target;
    } else {
        log "{1} ya tiene suficiente vida", target;
    }
}
```

Código 6.2: Implementación de una habilidad de curación condicional.